



Nazwa kwalifikacji: **Projektowanie, programowanie i testowanie aplikacji**

Symbol kwalifikacji: **INF.04**

Numer zadania: **01**

Wersja arkusza: **ZSK**

Wypełnia zdający

Numer zdającego*

--	--	--	--	--	--	--	--	--	--

Miejsce na naklejkę z
numerem zdającego i kodem
ośrodka

Czas trwania egzaminu: **180** minut

INF.04-01-25.12-ZSK

EGZAMIN ZAWODOWY

Rok 2025

CZĘŚĆ PRAKTYCZNA

**PODSTAWA PROGRAMOWA
2019**

Instrukcja dla zdającego

1. Na pierwszej stronie arkusza egzaminacyjnego wpisz w oznaczonym miejscu swój numer PESEL i naklej naklejkę z numerem PESEL i z kodem ośrodka.
2. Sprawdź, czy arkusz egzaminacyjny zawiera **6 stron** (nie licząc strony tytułowej) i nie zawiera błędów. Ewentualny brak stron lub inne usterki zgłoś przez podniesienie ręki przewodniczącemu zespołu nadzorującego.
3. Zapoznaj się z treścią zadania oraz stanowiskiem egzaminacyjnym. Masz na to 10 minut. Czas ten nie jest wliczany do czasu trwania egzaminu.
4. Czas rozpoczęcia i zakończenia pracy zapisze w widocznym miejscu przewodniczący zespołu nadzorującego.
5. Wykonaj samodzielnie zadanie egzaminacyjne. Przestrzegaj zasad bezpieczeństwa i organizacji pracy.
6. Po zakończeniu wykonania zadania pozostaw arkusz egzaminacyjny z rezultatami oraz KARTĘ OCENY na swoim stanowisku lub w miejscu wskazanym przez przewodniczącego zespołu nadzorującego.
7. Po uzyskaniu zgody zespołu nadzorującego możesz opuścić salę/miejsce przeprowadzania egzaminu.

Powodzenia!

* numer zdającego ma format: <klasa><numer z dziennika z zerem wiodącym><pierwsza litera nazwiska><pierwsza litera imienia>, np. dla ucznia Jan Kowalski z klasy 5F o numerze z dziennika 26 kod będzie miał postać: **5F26KJ**, a dla ucznia Adam Adamiak z klasy 5G o numerze z dziennika 1 kod będzie miał postać: **5G01AA**

Zadanie egzaminacyjne

UWAGA: numer, którym został podpisany arkusz egzaminacyjny jest w zadaniu nazywany numerem zdającego.

Wykonaj aplikację konsolową oraz webową według wskazań. Wykonaj testy do utworzonej aplikacji konsolowej zgodnie z opisem w części III instrukcji do zadania. Wykorzystaj konto Egzamin bez hasła.

Utwórz folder i nazwij go numerem zdającego. W folderze utwórz podfoldery: *konsolowa*, *webowa*, *testy*. Po wykonaniu każdej aplikacji, jej pełny kod (cały folder projektu) **spakuj do archiwum**. Następnie pozostaw w podfolderze jedynie skopiowane z projektu pliki źródłowe, których treść była modyfikowana, plik wykonywalny, jeśli jest to możliwe, oraz spakowane archiwum.

Część I. Aplikacja konsolowa

Napisz program sortujący tablicę metodą przez scalanie według zamieszczonej dokumentacji:

1. Podział zestawu danych na dwie równe części. Należy zwrócić uwagę, że w przypadku sortowania wektora (tablicy jednowymiarowej) podział nie ma sensu, bo wektor jest ze swej istoty "podzielony" - dostęp do każdego z jego elementów ma taki sam koszt.
2. Zastosowanie sortowania przez scalanie dla każdej z nich oddzielnie, chyba że pozostał już tylko jeden element.
3. Połączenie posortowanych podciągów w jeden posortowany ciąg.

Procedura scalania dwóch ciągów $A[1, \dots, n]$ i $B[1, \dots, m]$ do ciągu $C[1, \dots, m+n]$:

1. Utwórz wskaźniki na początki ciągów A i B $\rightarrow i = 1, j = 1$.
2. Jeżeli ciąg A wyczerpany ($i > n$), dołącz pozostałe elementy ciągu B do C i zakończ pracę.
3. Jeżeli ciąg B wyczerpany ($j > m$), dołącz pozostałe elementy ciągu A do C i zakończ pracę.
4. Jeżeli $A[i] < B[j]$ dołącz $A[i]$ do C i zwiększ i o jeden, w przeciwnym przypadku dołącz $B[j]$ do C i zwiększ j o jeden.
5. Powtarzaj od kroku 2 aż wszystkie wyrazy A i B trafią do C.

Założenia do programu:

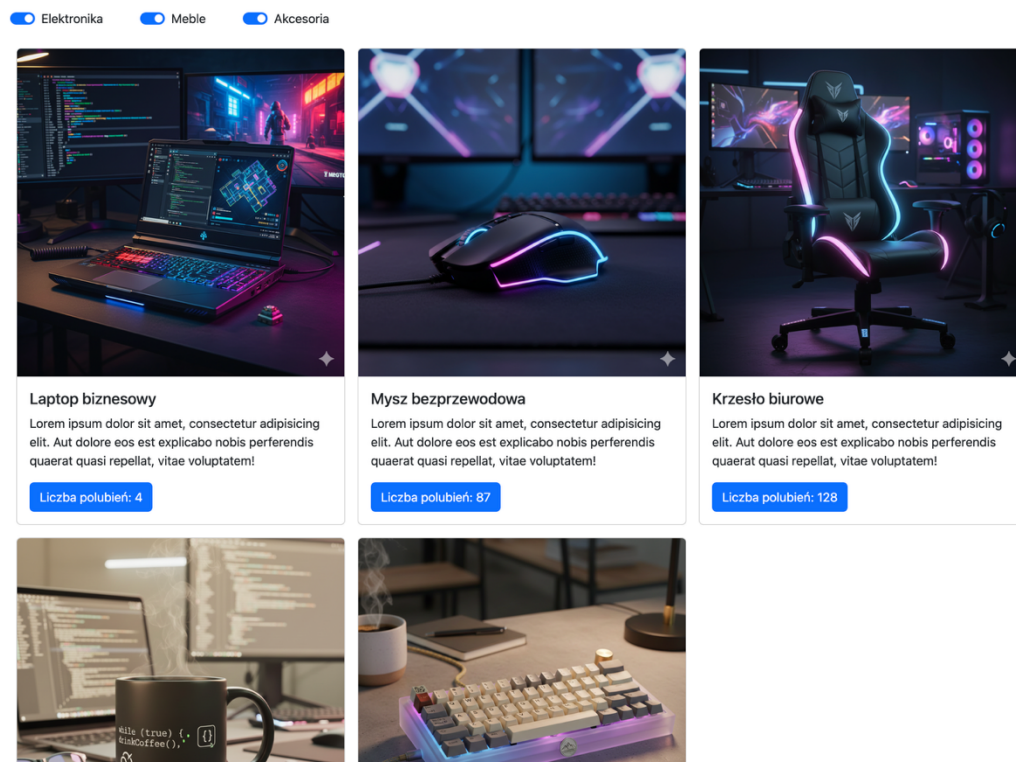
- Program wykonywany w konsoli.
- Zastosowano obiektowy język programowania zgodny z zainstalowanym na stanowisku egzaminacyjnym: C++ lub C# lub Java lub Python.
- Sortowanie odbywa się malejąco, nie wykorzystuje gotowych funkcji do sortowania oraz do szukania maksimum.
- Sortowana jest tablica 10 liczb całkowitych. Tablica jest polem klasy.
- Tablica jest wczytywana z klawiatury po uprzednim wypisaniu odpowiedniego komunikatu.
- Wszystkie elementy posortowanej tablicy są wyświetlane na ekranie.
- Klasa zawiera co najmniej dwie metody: scalania i dzielenia tablicy na 2 podtablice. Widzialność metody ogranicza się jedynie do klasy.
- Program powinien być zapisany czytelnie, z zasadami czystego formatowania kodu, należy stosować znaczące nazwy zmiennych i funkcji.
- Testowanie do programu wykonane zgodnie z wytycznymi z części III zadania egzaminacyjnego.

Kod aplikacji przygotuj do nagrania na płytę. W folderze konsolowa powinno znaleźć się archiwum całego projektu o nazwie *konsola.zip*, skopiowany z projektu plik z kodem źródłowym programu oraz plik wykonywalny, jeżeli istnieje.

Część II. Aplikacja webowa

Z zastosowaniem dostępnego na stanowisku egzaminacyjnym frameworka Angular lub biblioteki React.js wykonaj aplikację internetową typu front-end realizującą funkcję kategoryzacji produktów w galerii. Na obrazach 1, 2, 3 przedstawiono działanie aplikacji. W zależności od zastosowanego narzędzia wygląd aplikacji może nieznacznie się różnić. Na pulpicie znajduje się archiwum, z materiałami do wykonania zadania, o nazwie *pliki6.zip*, zabezpieczone hasłem: **P1ikiG4I3ri@**

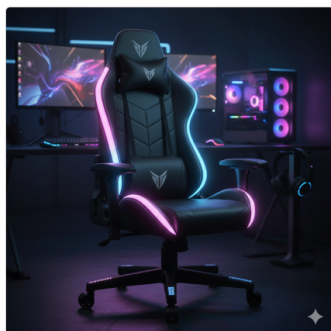
Galeria produktów



Obraz 1: Początkowy stan aplikacji webowej

Galeria produktów

☐ Elektronika ☒ Meble ☒ Akcesoria



Krzesło biurowe

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aut dolore eos est explicabo nobis perferendis quaerat quasi repellat, vitae voluptatem!

Liczba polubień: 4



Kubek firmowy

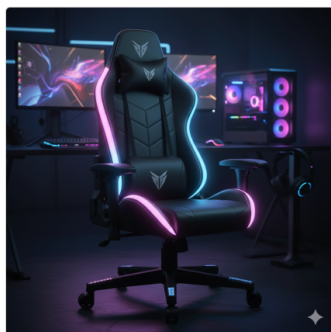
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aut dolore eos est explicabo nobis perferendis quaerat quasi repellat, vitae voluptatem!

Liczba polubień: 87

Obraz 2: Stan aplikacji po zastosowaniu filtrów

Galeria produktów

☐ Elektronika ☒ Meble ☒ Akcesoria



Krzesło biurowe

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aut dolore eos est explicabo nobis perferendis quaerat quasi repellat, vitae voluptatem!

Liczba polubień: 4



Kubek firmowy

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aut dolore eos est explicabo nobis perferendis quaerat quasi repellat, vitae voluptatem!

Liczba polubień: 91

Obraz 3: Stan aplikacji po kilkukrotnym wciśnięciu przycisku „Liczba polubień: X” na produkcie „Kubek firmowy”

Założenia aplikacji

- Aplikacja składa się z komponentu **App**, wyświetlanego po uruchomieniu oraz komponentu **ProductCard**, który przedstawia pojedynczy produkt. Widok początkowy aplikacji przedstawiony jest na Obrazie 1 (produkty mogą być w dowolnej kolejności)
- Do utworzenia aplikacji należy wykorzystać zdjęcia oraz plik *products.js* (dla wersji Javascript) lub *products.ts* (dla wersji Typescript) wypakowane z archiwum
- Obrazy należy umieścić w folderze *assets* (egzamin/src/assets lub egzamin/public/assets)
- Plik *products.js* oraz plik *products.ts* zawierają listę produktów. Każdy obiekt zawiera pola:
 - id
 - filename
 - name
 - category
 - description
 - likes
- Wymagania dotyczące komponentu **App**:
 - Dla głównego bloku komponentu zastosowano klasy *container py-5*
 - Wewnątrz głównego bloku znajdują się dwa bloki o klasie *row*
 - Pierwszy blok o klasie *row* zawiera:
 - Blok o klasach *col-12 my-3 p-0* wewnątrz którego znajduje się nagłówek pierwszego stopnia o zawartości „Galeria produktów”
 - Blok o klasach *col-12 d-flex gap-5 p-0 my-3* wewnątrz którego znajdują się 3 pola *switch* o etykietach: Elektronika, Meble, Akcesoria. Domyślnie wszystkie pola *switch* powinny być zaznaczone
 - Drugi blok o klasie *row* zawiera listę produktów pofiltrowaną zgodnie z zaznaczanymi polami typu *switch*
- Wymagania dotyczące komponentu **ProductCard**:
 - Dla głównego bloku komponentu zastosowano klasy *col-12 col-md-4 py-2*
 - Wewnątrz głównego komponentu znajduje się blok implementujący kartę (card) z biblioteki Bootstrap
 - Kliknięcie przycisku „Liczba polubień: X” zwiększa liczbę polubień danego produktu o 1
- Pola *switch* stylowane są zgodnie z przykładami w tabeli 1
- W stylowaniu elementu *body* aplikacji zastosowano brak marginesów
- W stanie początkowym włączone są wszystkie pola *switch*, co powoduje wyświetlenie wszystkich produktów
- Wyświetlane są tylko zdjęcia z kategorii, dla której jest włączone pole *switch* (obraz 2)
- W aplikacji zastosowano pętle oraz warunki do wyświetlenia bloków zdjęć. Aplikacja jest napisana uniwersalnie i działa poprawnie też dla innej liczby zdjęć
- Aplikacja powinna być zapisana czytelnie, z zachowaniem zasad czystego formatowania kodu, należy stosować znaczące nazwy zmiennych i funkcji.

Podjmij próbę uruchomienia aplikacji w przeglądarce internetowej. Wykonaj zrzut ekranu potwierdzający próbę uruchomienia aplikacji. Zrzut powinien obejmować cały obszar ekranu z widocznym paskiem zadań i nazywać się *webowa.jpg*. Zrzut umieść w folderze *webowa*.

Kod aplikacji przygotuj do nagrania na płytę. W folderze *webowa* powinno znaleźć się archiwum całego folderu projektu o nazwie *web.zip* oraz pliki z kodem źródłowym, które były modyfikowane przez zdającego.

Część III. Testy jednostkowe aplikacji konsolowej

Wykonaj test aplikacji konsolowej. Test powinien być fragmentem kodu, który należy umieścić w programie głównym aplikacji. Test zapisz w folderze *testy*, w pliku o nazwie test, z rozszerzeniem charakterystycznym dla aplikacji pisanych w danym języku programowania.

W teście sprawdź działanie funkcji sortującej dla wylosowanych 10 liczb pseudolosowych. Liczby powinny być całkowite dodatnie z przedziału [1,1000]. Uruchom aplikację z testem i udokumentuj to zrzutem ekranu. Jeżeli aplikacja nie kompiluje się, na zrzucie umieść środowisko programistyczne z oknem kompilacji. Zrzut powinien obejmować cały obszar ekranu z widocznym paskiem zadań i nazywać się *konsola_testy.jpg*. Zrzut umieść w folderze *testy*.

*UWAGA: Nagraj płytę z rezultatami pracy. W folderze z numerem zdającego powinny się znajdować podfoldery *testy*, *konsolowa*, *webowa*. W folderze *testy*: pliki ze zrzutami oraz plik *test*. W folderze *konsolowa*: spakowany cały projekt aplikacji konsolowej, źródło, plik wykonywalny, jeżeli istnieje. W folderze *webowa*: spakowany cały projekt aplikacji web, pliki ze źródłami komponentu, *html*, *css* i inne modyfikowane pliki. Opisz płytę numerem zdającego i pozostaw na stanowisku, zapakowaną w pudełku wraz z arkuszem egzaminacyjnym.*

Czas przeznaczony na wykonanie zadania wynosi 180 minut.

Ocenie będą podlegać 4 rezultaty

- implementacja, kompilacja, uruchomienie programu,
- aplikacja konsolowa,
- aplikacja web,
- testy do aplikacji konsolowej.

Tabela 1. Wybrane elementy biblioteki Bootstrap

Angular To use Bootstrap add to styles.css: @import "~bootstrap/dist/css/bootstrap.css";
React.js To use Bootstrap add: import 'bootstrap/dist/css/bootstrap.css';
Bootstrap Switch A switch has the markup of a custom checkbox but uses the <i>.form-switch</i> class to render a toggle switch. Consider using <i>role="switch"</i> to more accurately convey the nature of the control to assistive technologies that support this role. In older assistive technologies, it will simply be announced as a regular checkbox as a fallback. Switches also support the disabled attribute. <pre> <div class="form-check form-switch"> <input class="form-check-input" type="checkbox" role="switch" id="switchCheckDefault"> <label class="form-check-label" for="switchCheckDefault"> Default switch checkbox input </label> </div> <div class="form-check form-switch"> <input class="form-check-input" type="checkbox" role="switch" id="switchCheckChecked" checked> <label class="form-check-label" for="switchCheckChecked"> Checked switch checkbox input </label> </div> </pre>
Bootstrap Card Cards are built with as little markup and styles as possible, but still manage to deliver a ton of control and customization. Built with flexbox, they offer easy alignment and mix well with other Bootstrap components. They have no margin by default, so use spacing utilities as needed. <pre> <div class="card"> <div class="card-body"> <h5 class="card-title">Card title</h5> <p class="card-text"> Some quick example text to build on the card title and make up the bulk of the card's content. </p> Go somewhere </div> </div> </pre>